Session 1

Introduction to JSP

J2EE Web Apps

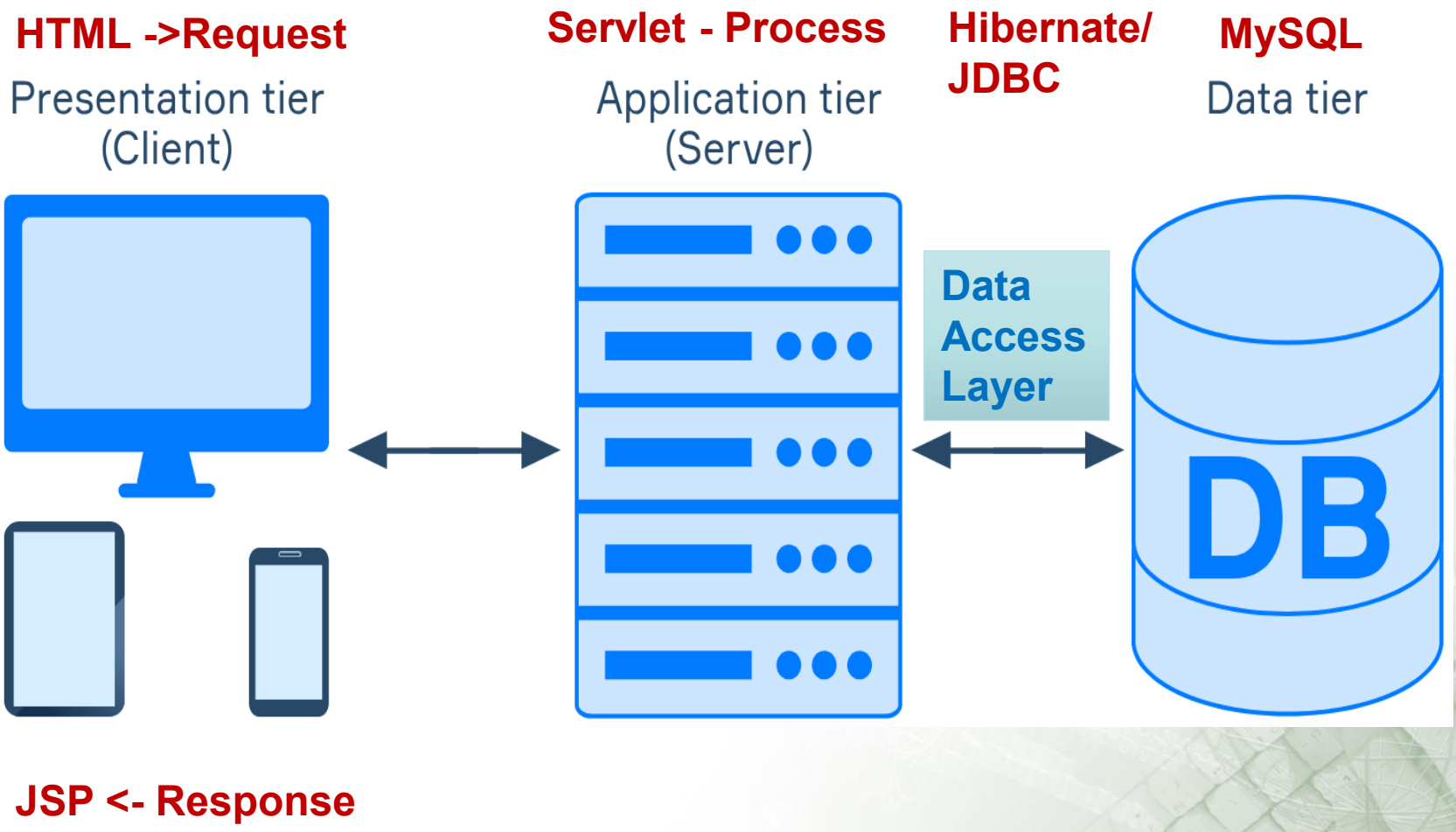
Review

- JavaMail is a set of abstract classes.
- JavaMail is platform and protocol independent and helps in sending and receiving mail messages.
- `Connect()` method of the store object is used to connect to view folders and messages.
- Multipart message is an object of the `Message` class. The content type is set to multipart.
- `Session` is the highest level class which cannot be subclassed.
- A bodypart can be plain text as well as an attachment.
- Multipart message can have more than one bodypart.

Objectives

- *Explain JSP*
- *Identify the advantages of using JSP*
- *Describe various elements of JSP*
- *Describe the JSP Life Cycle*
- *Develop JSP using Eclipse/NetBeans IDE*

J2EE Web App Pattern



Introduction to JSP

- It is a server-side technology basically used for creating web applications.
- ***This technology creates web content which consists of both dynamic as well as static components.***
- A JSP page is a text document. It contains two types of text: *static content and dynamic content*.
- Static content can be expressed in any text-based format, say, HTML.
- Whereas, the dynamic content comprises of the Java code.

Java Server Pages

- JavaServer Pages (JSP) is a server-side programming technology that enables the creation of dynamic, platform-independent method for building Web-based applications.
- JSP have access to the entire family of Java APIs, including the JDBC API to access enterprise databases.
- JavaServer Pages (JSP) is a technology for developing web pages that support dynamic content which helps developers insert java code in HTML pages by making use of special JSP tags, most of which start with <% and end with %>.

Java Server Pages

- A Java Server Pages component is a type of Java servlet that is designed to fulfill the role of a user interface for a Java web application.
- Web developers write JSPs as text files that combine HTML or XHTML code, XML elements, and embedded JSP actions and commands.
- Using JSP, you can collect input from users through web page forms, present records from a database or another source, and create web pages dynamically.

Why Use JSP?

- Performance is significantly better because JSP allows embedding Dynamic Elements in HTML Pages itself.
- JSP are always compiled before it's processed by the server unlike CGI/Perl which requires the server to load an interpreter and the target script each time the page is requested.
- JavaServer Pages are built on top of the Java Servlets API, so like Servlets, JSP also has access to all the powerful Enterprise Java APIs, including JDBC, JNDI, EJB, JAXP etc.

Why Use JSP?

- JSP pages can be used in combination with servlets that handle the business logic, the model supported by Java servlet template engines.
- JSP is an integral part of Java EE, a complete platform for enterprise class applications.
- This means that JSP can play a part in the simplest applications to the most complex and demanding.

Advantages of JSP

- **vs. Active Server Pages (ASP):** The advantages of JSP are twofold. First, the dynamic part is written in Java, not Visual Basic or other MS specific language, so it is more powerful and easier to use. Second, it is portable to other operating systems and non-Microsoft Web servers.
- **vs. Pure Servlets:** It is more convenient to write (and to modify!) regular HTML than to have plenty of println statements that generate the HTML.

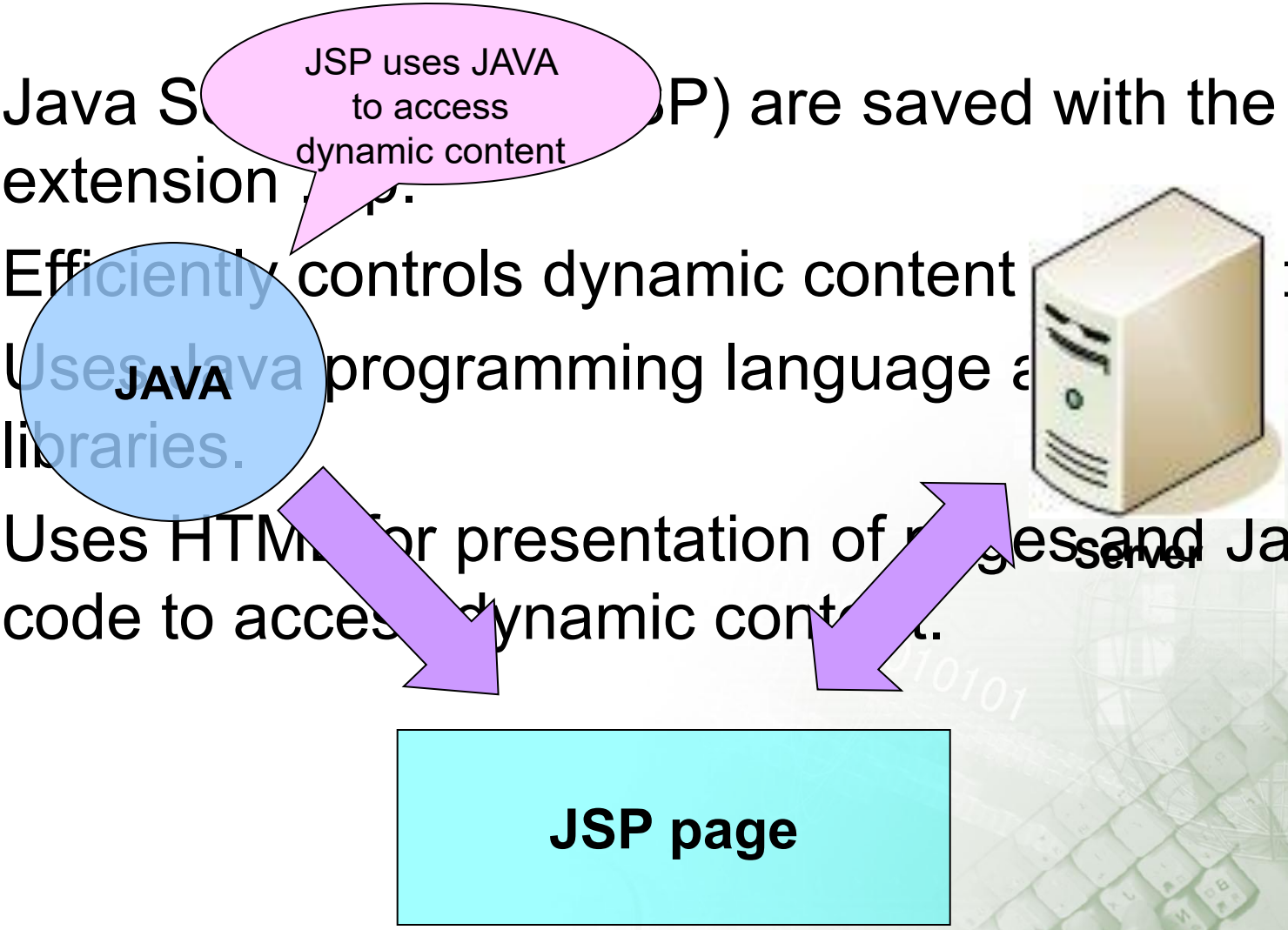
Advantages of JSP

- **vs. JavaScript:** JavaScript can generate HTML dynamically on the client but can hardly interact with the web server to perform complex tasks like database access and image processing etc.

- **vs. Static HTML:** Regular HTML, of course, cannot contain dynamic information.

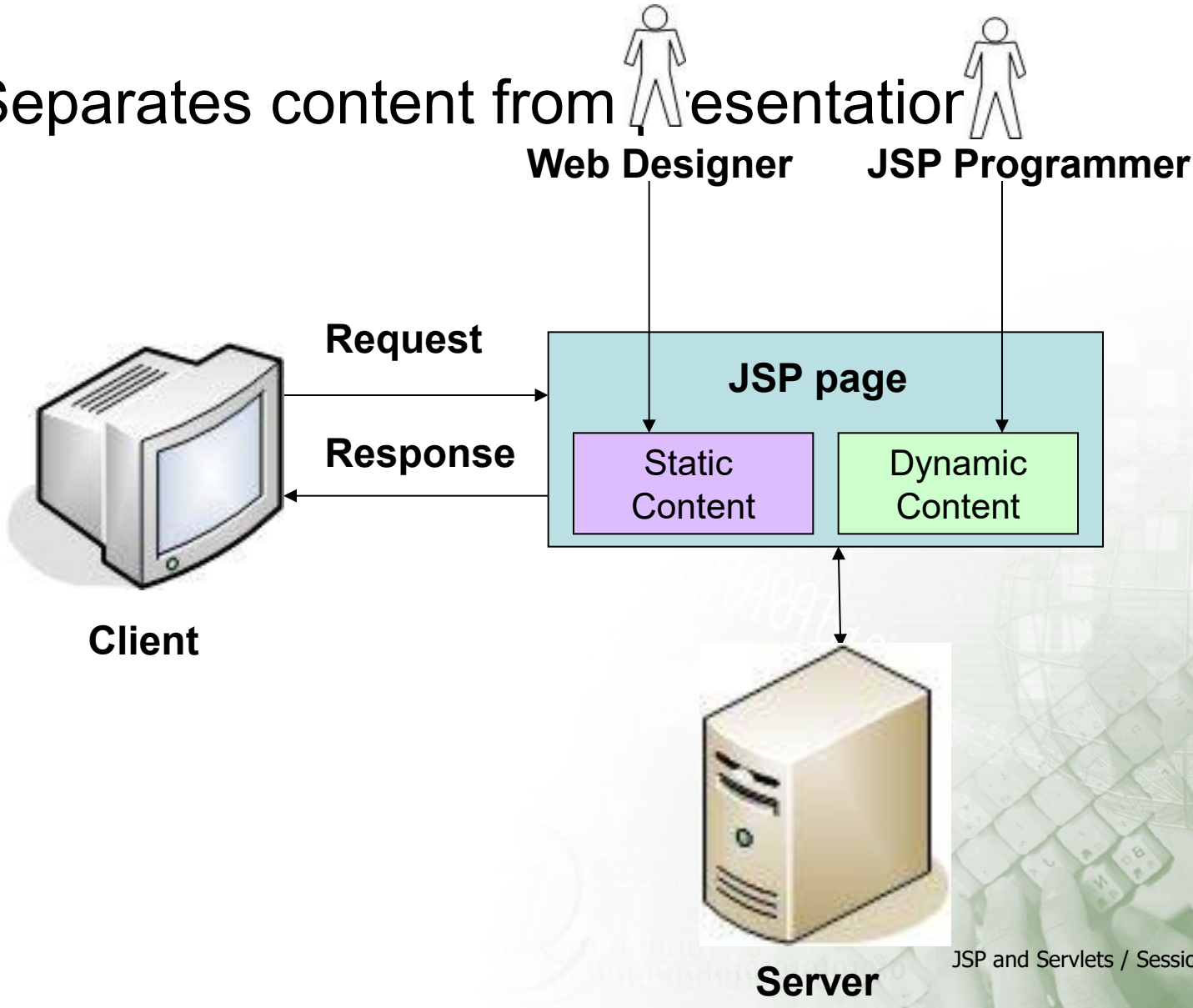
Introduction to JSP

- Java S…SP) are saved with the extension .jsp.
- Efficiently controls dynamic content …on.
- Uses Java programming language a… libraries.
- Uses HTM… for presentation of …es and Java code to acces… dynamic cont…t.



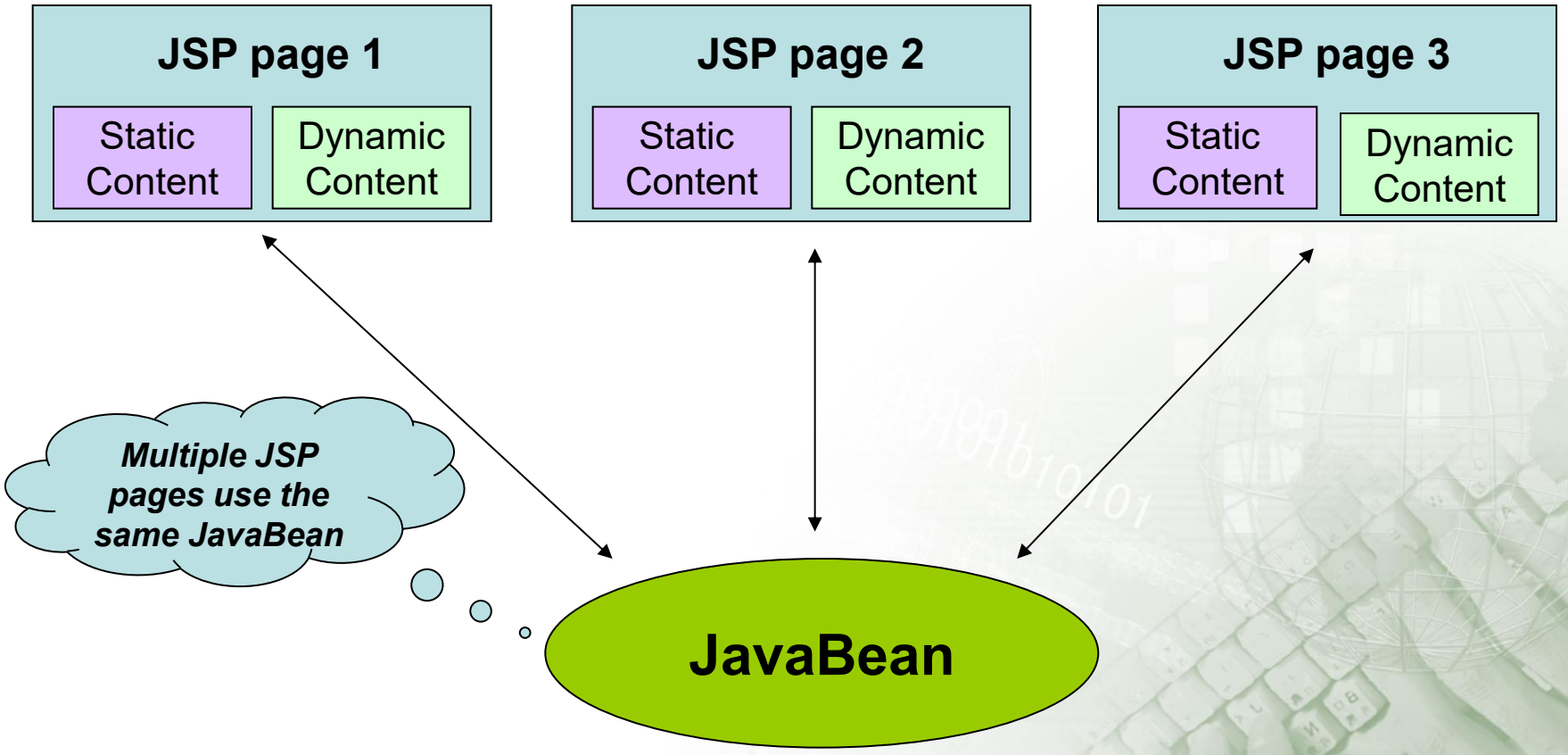
Benefits of JSP 3-1

- Separates content from presentation



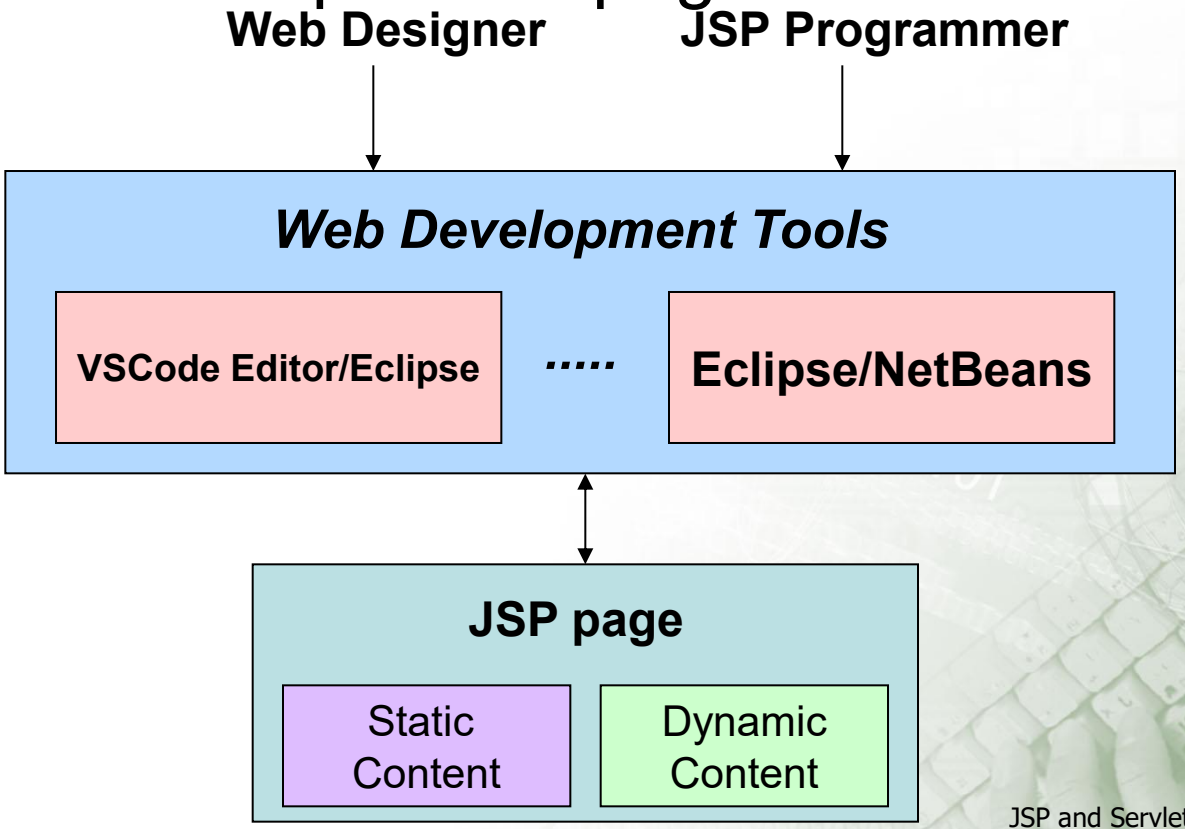
Benefits of JSP 3-2

- Emphasizes reusable components



Benefits of JSP 3-3

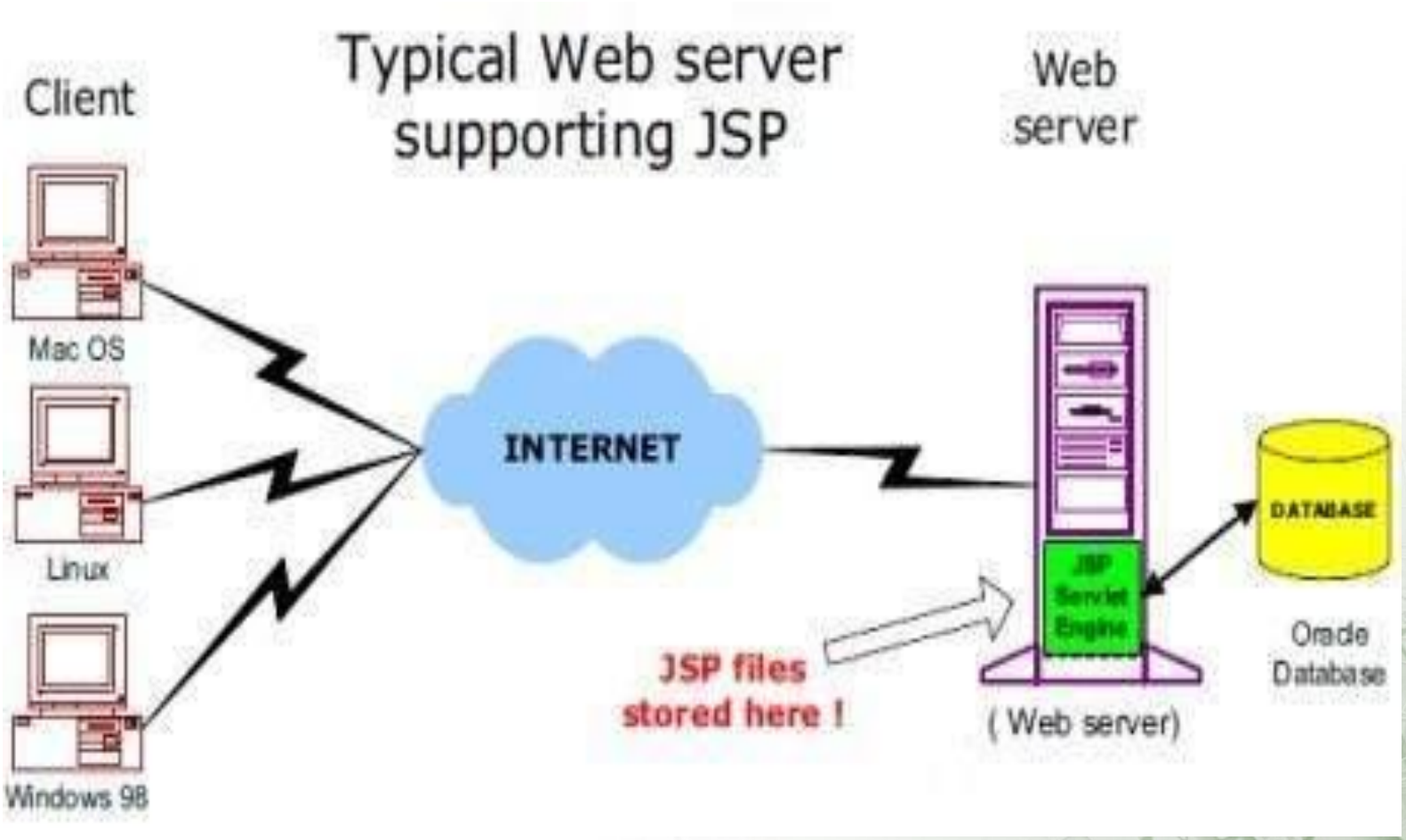
- Simplified page development - Web designer and Web programmer use Web development tools to develop a JSP page.



JSP - Architecture

- The web server needs a JSP engine ie. container to process JSP pages.
- The JSP container is responsible for intercepting requests for JSP pages.
- A JSP container works with the Web server to provide the runtime environment and other services a JSP needs.
- It knows how to understand the special elements that are part of JSPs.

JSP container and JSP files in a Web Application



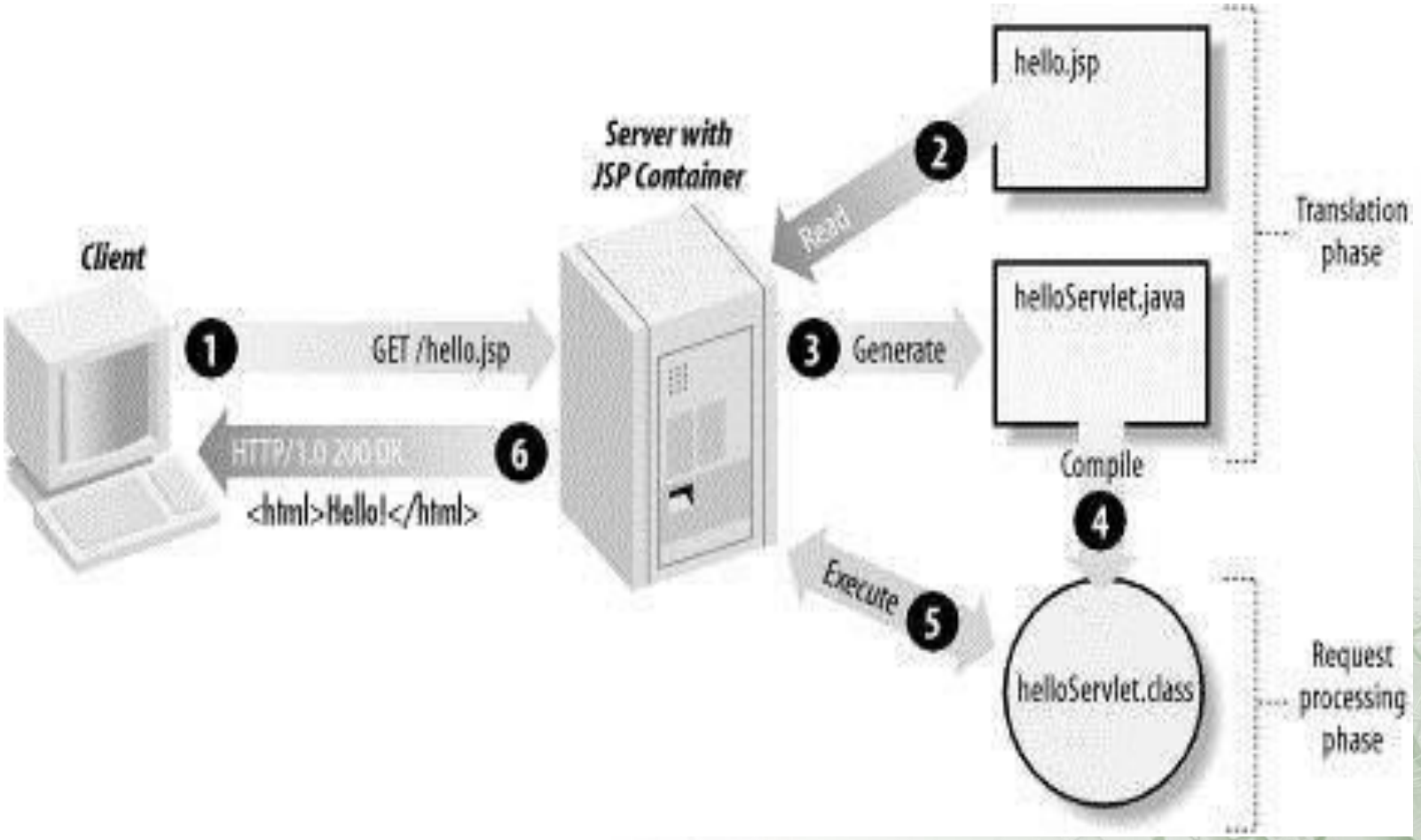
JSP Processing

- As with a normal page, your browser sends an HTTP request to the web server.
- The web server recognizes that the HTTP request is for a JSP page and forwards it to a JSP engine.
- This is done by using the URL or JSP page which ends with **.jsp** instead of .html.
- The JSP engine loads the JSP page from disk and converts it into a servlet content.
- This conversion is very simple in which all template text is converted to println( ) statements and all JSP elements are converted to Java code that implements the corresponding dynamic behavior of the page.

JSP Processing

- The JSP engine compiles the servlet into an executable class and forwards the original request to a servlet engine.
- A part of the web server called the servlet engine loads the Servlet class and executes it.
- During execution, the servlet produces an output in HTML format, which the servlet engine passes to the web server inside an HTTP response.
- The web server forwards the HTTP response to your browser in terms of static HTML content.
- Finally web browser handles the dynamically generated HTML page inside the HTTP response exactly as if it were a static page.

JSP Processing



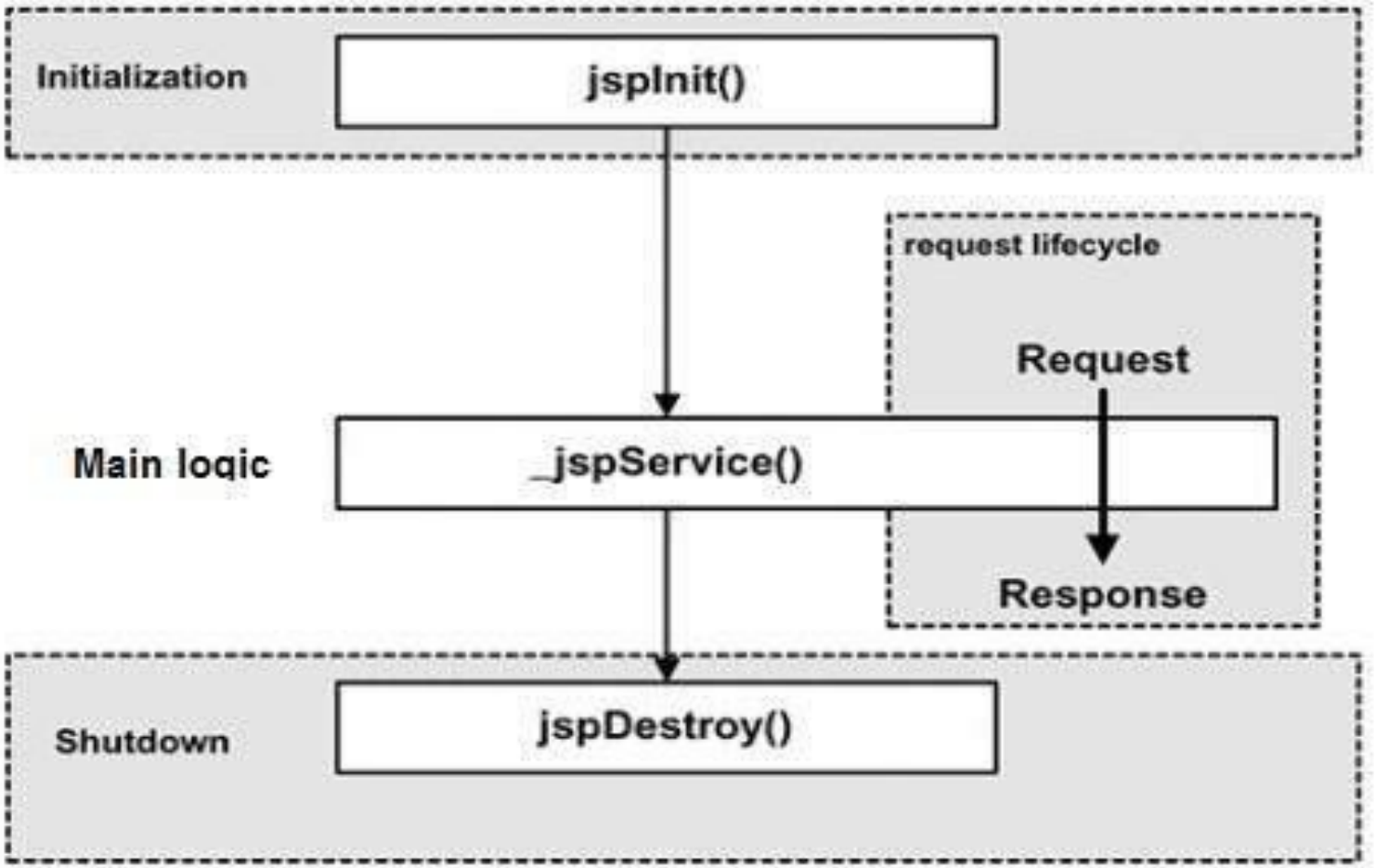
JSP Processing

- Typically, the JSP engine checks to see whether a servlet for a JSP file already exists and whether the modification date on the JSP is older than the servlet.
- If the JSP is older than its generated servlet, the JSP container assumes that the JSP hasn't changed and that the generated servlet still matches the JSP's contents.
- This makes the process more efficient than with other scripting languages (such as PHP) and therefore faster.
- So in a way, a JSP page is really just another way to write a servlet without having to be a Java programming wiz.
- Except for the translation phase, a JSP page is handled exactly like a regular servlet

JSP - Life Cycle

- A JSP life cycle can be defined as the entire process from its creation till the destruction .
  - Compilation
  - Initialization
  - Execution
  - Cleanup

Phases of JSP life cycle



JSP Compilation

- When a browser asks for a JSP, the JSP engine first checks to see whether it needs to compile the page.
- If the page has never been compiled, or if the JSP has been modified since it was last compiled, the JSP engine compiles the page.

The compilation process involves three steps:

- Parsing the JSP.
- Turning the JSP into a servlet.
- Compiling the servlet.

JSP Initialization

- When a container loads a JSP it invokes the jspInit() method before servicing any requests.
- If you need to perform JSP-specific initialization, override the jspInit() method:

```
public void jspInit(){ // Initialization code... }
```

- Typically initialization is performed only once and as with the servlet init method, you generally initialize database connections, open files, and create lookup tables in the jspInit method.

JSP Execution

- This phase of the JSP life cycle represents all interactions with requests until the JSP is destroyed.
- Whenever a browser requests a JSP and the page has been loaded and initialized, the JSP engine invokes the **_jspService()** method in the JSP.

JSP Cleanup

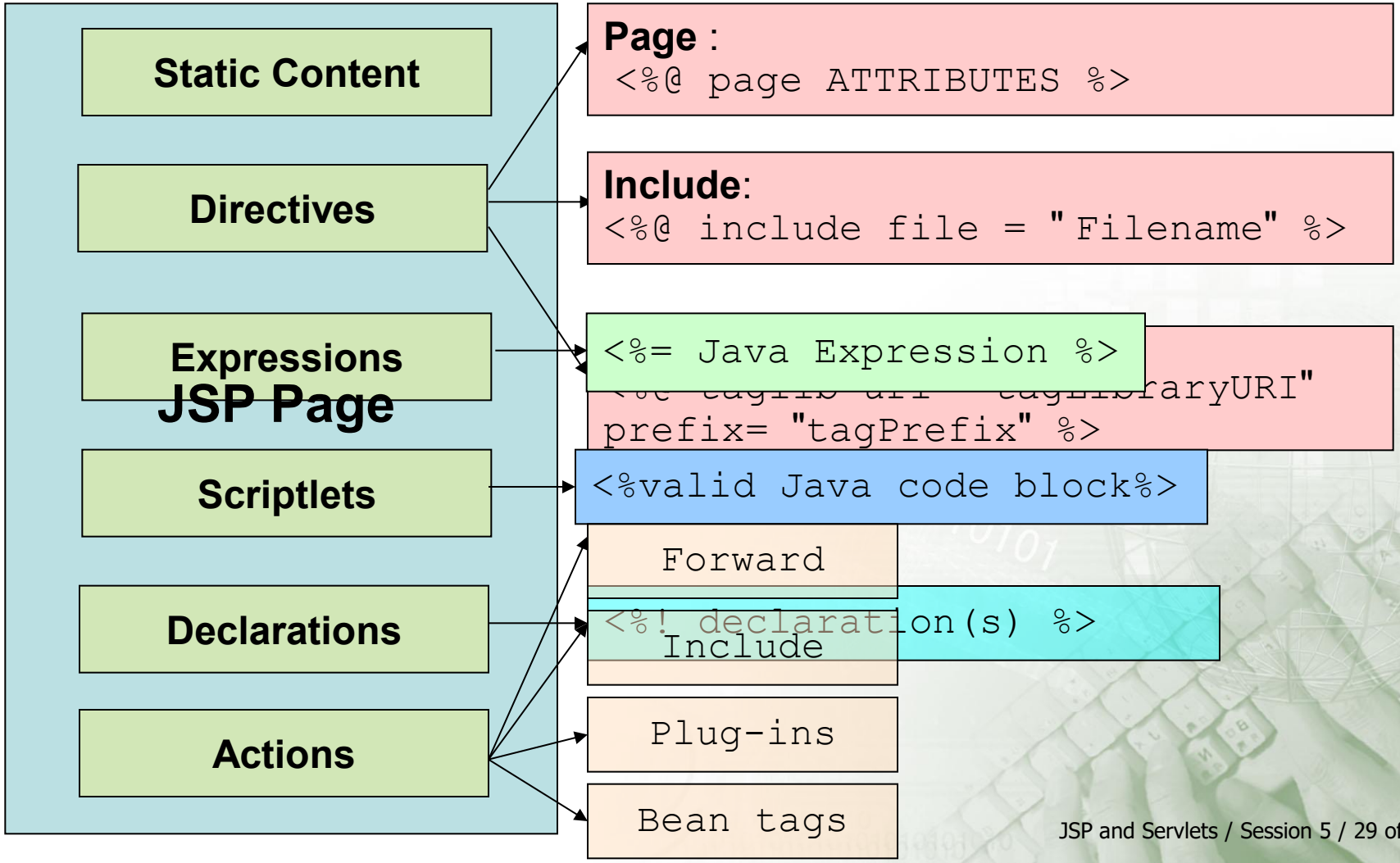
- The destruction phase of the JSP life cycle represents when a JSP is being removed from use by a container.
- The **jspDestroy()** method is the JSP equivalent of the destroy method for servlets.
- Override jspDestroy when you need to perform any cleanup, such as releasing database connections or closing open files.

Advantages of JSP Over Servlets

JSP	Servlets
Extension to Servlet	Not an extension to servlet
Easy to Maintain	Bit complicated
No need to recompile or redeploy	The code needs to be recompiled
Less code than a servlet	More code compared to JSP

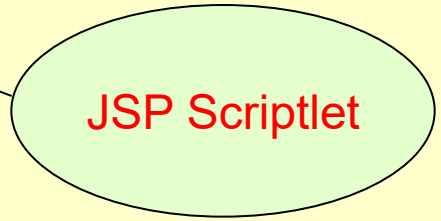
Elements of JSP 2-1

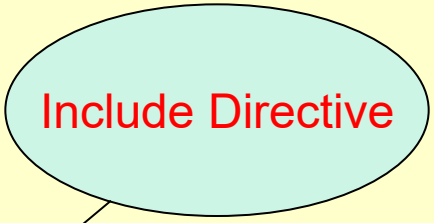
- Elements of a JSP page



Elements of JSP 2-2

```
<%
    if (Calendar.getInstance().get(Calendar.AM_PM)
    == Calendar.AM)
    {
%>
Good Morning
<%
    }
    else
    {
%>
    Good Afternoon
<%
    }
%>
<%@ include file="copyrightfile.html" %>
…
…
</html>
```





Static Content

- Static content is the text written on a Web page.
- Any text-based format can be used to write static


```
<html>
    <%@  page contentType = "text/html" %>
    <head>
        <title>Example for static content</title>
    </head>
    …
    //Contains text in specified content type.
    …
    </body>
</html>
```

JSP Directives

- A JSP directive affects the overall structure of the servlet class.
- It usually has the following form:

```
<%@ directive attribute="value" %>
```

Directive	Description
<%@ page ... %>	Defines page-dependent attributes, such as scripting language, error page, and buffering requirements.
<%@ include ... %>	Includes a file during the translation phase.
<%@ taglib ... %>	Declares a tag library, containing custom actions, used in the page

JSP Directives

```
<html>
...
    <%@ page language="Java" import=
    "java.rmi.*, java.util.*"
```

```
<html>
    <head>
        <title>Testing include
        directive</title>
    </head>
```

```
// testFile.html
<html>
<body>
The JSP <b>"include"</b> directive example.
</body>
</html>
```

```
...
</html>
```

sing of

page

include Directive

age Directive

Demonstration: Example 1

JSP Expression

- A JSP expression element contains a scripting language expression that is evaluated, converted to a String, and inserted where the expression appears in the JSP file.
- Because the value of an expression is converted to a String, you can use an expression within a line of text, whether or not it is tagged with HTML, in a JSP file.
- The expression element can contain any expression that is valid according to the Java Language Specification but you cannot use a semicolon to end an expression.

Syntax of JSP Expression

```
<%= expression %>
```

```
<html>
 <head><title>A Comment Test</title></head>
<body> <p> Today's date:
<%= (new java.util.Date()).toLocaleString()%>
 </p>
</body>
 </html>
```

JSP Expression

- Contains a Java statement

```
<html>
    <head>
        <title>Testing Expression directive</title>
    </head>
    <body>
        <h1> Testing Expression directive </h1>
        <% int i = 2, j=3;%>
        <% i++;%>
        <% j=j+i; %>
        …
    </body>
</html>
```

JSP Expression

JSP Scriptlet

- A scriptlet can contain any number of JAVA language statements, variable or method declarations, or expressions that are valid in the page scripting language.

```
<html>
 <head><title>Hello World</title></head>
 <body> Hello World!<br/>
<% out.println("Your IP address is " +
request.getRemoteAddr()); %>
</body>
 </html>
```

JSP Scriptlet

```
<html>
    <head>
        <title>Scriptlet of a JSP </title>
    </head>
    <body>
        <h1>Scriptlet of a JSP</h1>
        <%
            int k=0;
            for(int i=0;i<5;i++)
            {
                k=k+i;
                out.println(k);
            }
        %>
        ...
    </body>
</html>
```

JSP Scriptlet

JSP Declarations

- A declaration declares one or more variables or methods that you can use in Java code later in the JSP file.
- You must declare the variable or method before you use it in the JSP file.

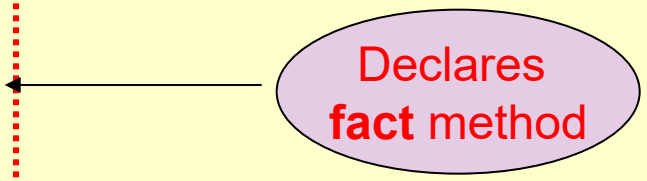
```
<%! declaration; [ declaration; ]+ ... %>
```

```
<%! int i = 0; %>
<%! int a, b, c; %>
<%! Circle a = new Circle(2.0); %>
```

JSP Declaration

- Used to define variables and methods in a JSP page.

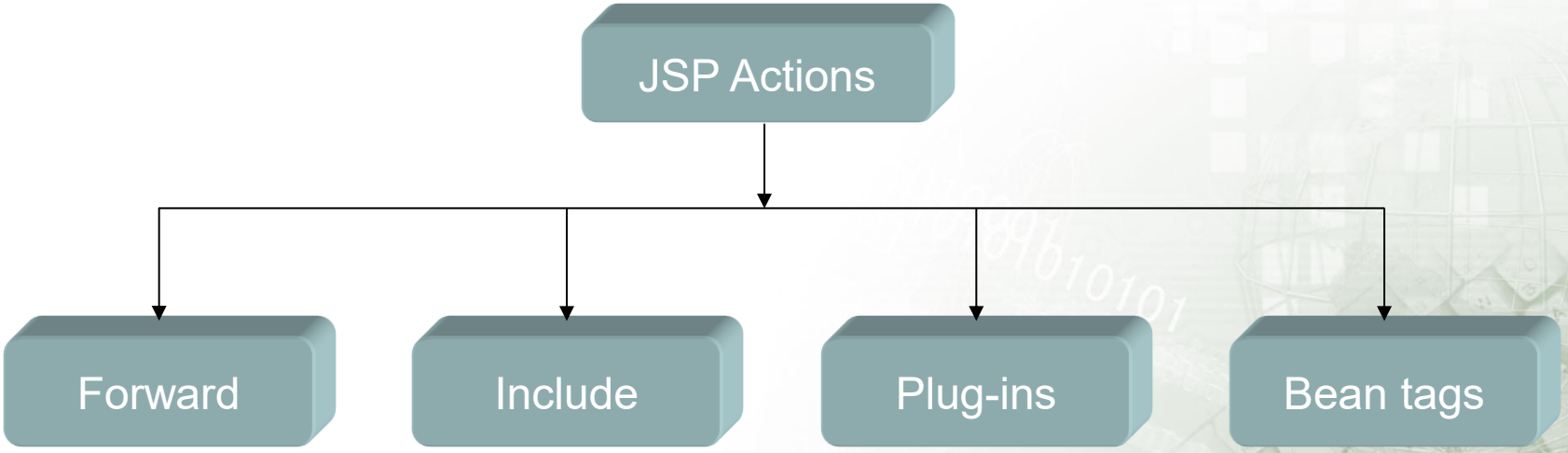
```
<%!
public long fact (long x)
{
if (x == 0) return 1;
else return x * fact(x-1);
}
%>
```




```
<h2>The Factorial of number is : <%= fact(10) %> </h2>
```

JSP Actions

- JSP actions use constructs in XML syntax to control the behavior of the servlet engine.
- Allows the transfer of control between pages
- Allows JSP pages to access JavaBeans component objects stored on the server.



Action element Syntax-

```
<jsp:action_name attribute="value" />
```

Syntax	Purpose
jsp:include	Includes a file at the time the page is requested
jsp:useBean	Finds or instantiates a JavaBean
jsp:setProperty	Sets the property of a JavaBean
jsp:getProperty	Inserts the property of a JavaBean into the output
jsp:forward	Forwards the requester to a new page
jsp:plugin	Generates browser-specific code that makes an OBJECT or EMBED tag for the Java plugin

Common Attributes

Id attribute:

- The id attribute uniquely identifies the Action element, and allows the action to be referenced inside the JSP page.
- If the Action creates an instance of an object the id value can be used to reference it through the implicit object PageContext.

Scope attribute:

- This attribute identifies the lifecycle of the Action element.
- The id attribute and the scope attribute are directly related, as the scope attribute determines the lifespan of the object associated with the id.
- The scope attribute has four possible values: (a) page, (b)request, (c)session, and (d) application.

The <jsp:forward> Action

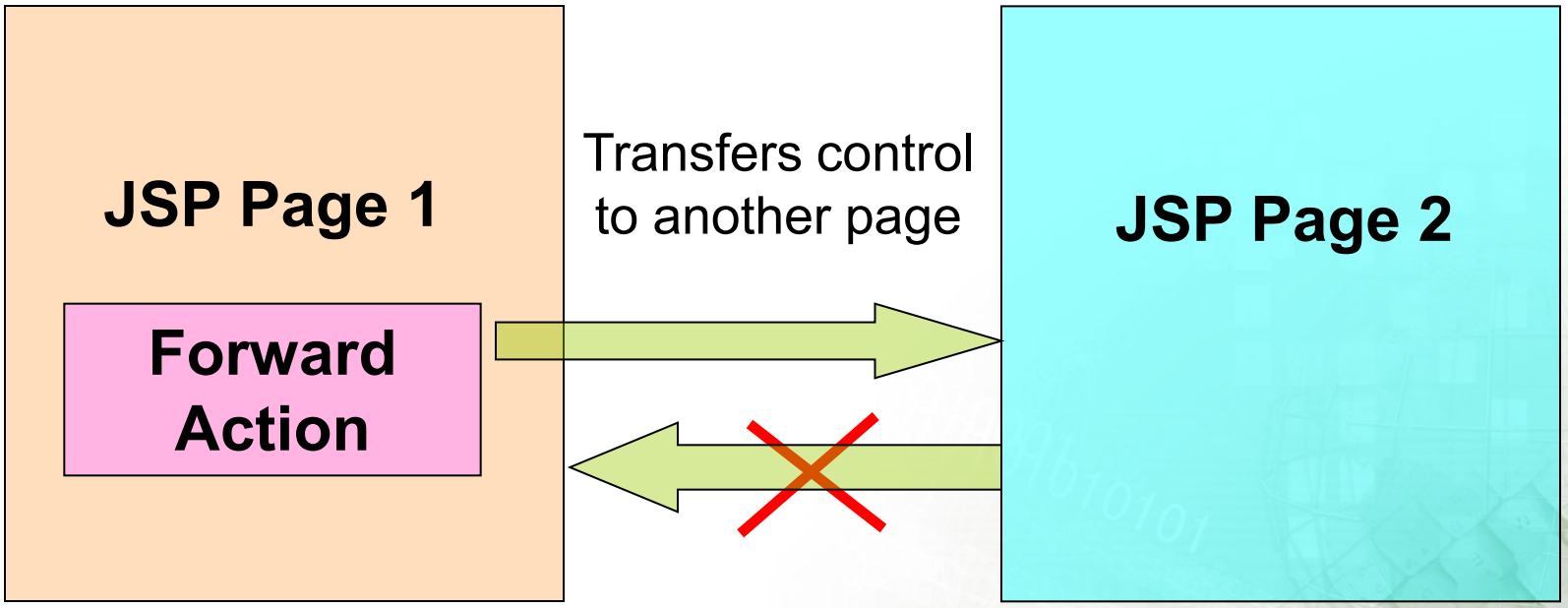
- The **forward** action terminates the action of the current page and forwards the request to another resource such as a static page, another JSP page, or a Java Servlet.

- Syntax

```
<jsp:forward page="Relative URL" />
```

Forward Action

- Permanently transfers control from a JSP page to another location.



<jsp:include> Action

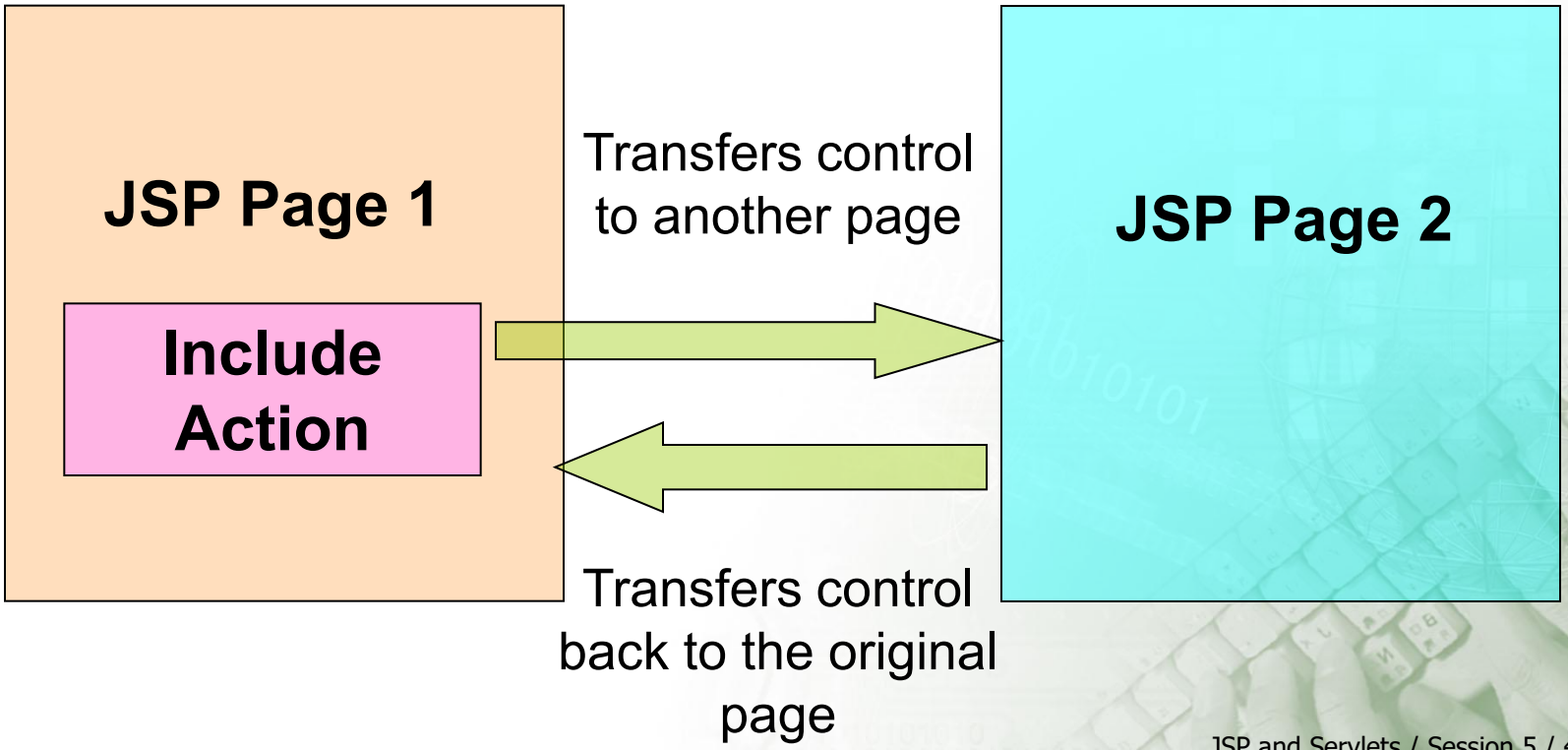
- This action lets you insert files into the page being generated.
- The syntax looks like this:

```
<jsp:include page="relative URL" flush="true" />
```

- Unlike the **include** directive, which inserts the file at the time the JSP page is translated into a servlet, this action inserts the file at the time the page is requested.

Include Action 2-1

- Inserts the content generated by remote resource in the output of the current JSP page.
- Temporarily transfers the control to a new page



Include Action 2-2

```
 static private String[] articles = { "a ", "an " };
                public String withArticle(String noun) {
                    if (startsWithVowel(noun)) return
articles[1] + noun;
                    else return articles[0] + noun;
                }
        %>
    </head>
    <body bgcolor="#ffffff">
        <br />
        <b>Starts with a vowel: </b>
        <%=startsWithVowel("I am a ve                    %>
<br/>
        <b>String entered is: </b>
        <%=word%>
        <br />
        <b>Article:</b>
        <%=withArticle("apple")%> <br />
    </body>
</html>
                }
```

JSP Declarations

Demonstration: Example 4

JSP Plug-ins and Bean Tags

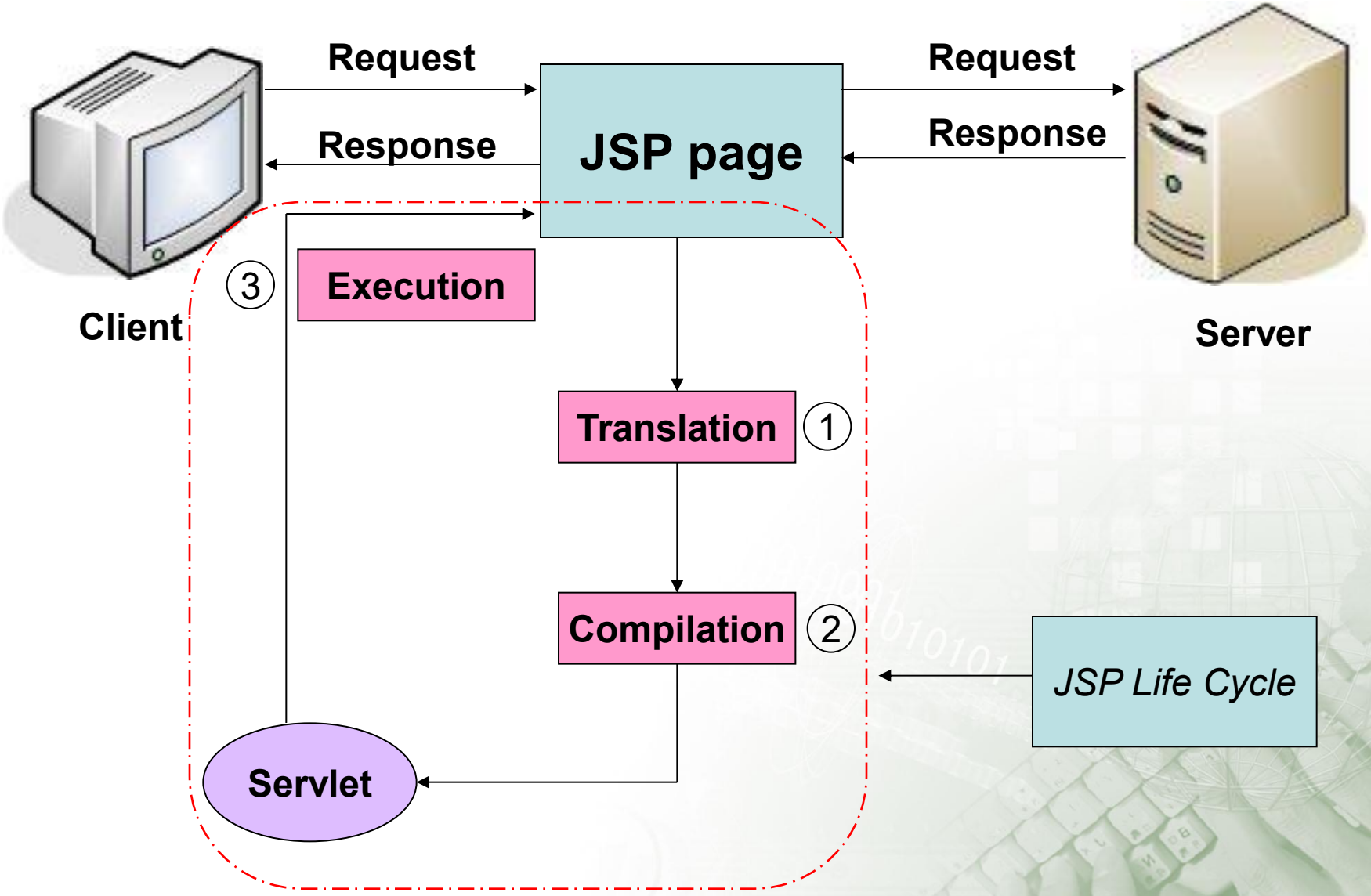
- JSP Plug-ins - Used to generate browser-specific HTML.
- JSP Bean Tags – Used to interact with JavaBeans stored on the server.

Displaying applet in JSP

- The **jsp:plugin** action tag is used to embed applet in the jsp file.
- The jsp:plugin action tag downloads plugin at client side to execute an applet or bean.
- Applet is java program that can be embedded into HTML pages.
- Java applets runs on the java enables web browsers such as mozila and internet explorer.

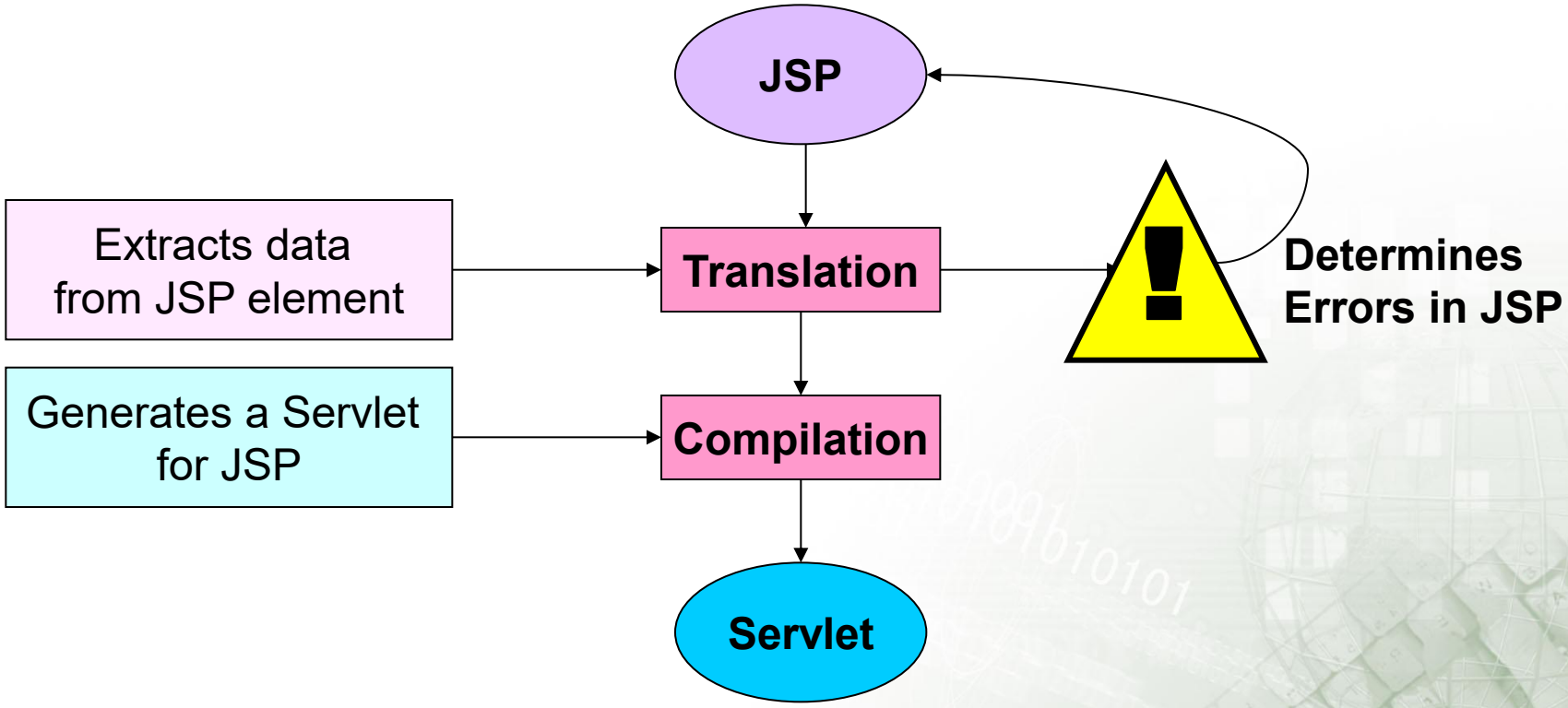
```
<jsp:plugin type= "applet | bean" code= "nameOfClassFile"
codebase= "directoryNameOfClassFile" > ……..
  [<jsp:params>
  <jsp:param name="parametername"
  value="parametervalue" /> …… </jsp:params>]
</jsp:plugin>
```

JSP Life Cycle 3-1



JSP Life Cycle 3-2

- Translation and Compilation



JSP Life Cycle 3-3

- Execution – Actions performed during execution are:

Buffering Output

```
<%@ page buffer = "none|20kb" %>
```

Sets buffer size

Handling Errors

```
<%@ page errorPage = "errorJSP.html" %>
```

Specifies error page

JSP Application in Eclipse

```
<html>
    <head>
    <meta http-equiv="Content-Type" content="text/html;
charset=UTF-8">
    <title>Declaration Tag - Methods</title>
    </head>
    <h3>Calculating area and circumference of a
Circle</h3>
    <hr/>
    <b>Radius of circle:</b> <%=radius%> cm<br/>
    <b>Diameter:</b> <%=getDiameter()%> cm<br/>
    <b>Area of Circle is:</b> <%=getArea()%>
    cm<sup>2</sup><br/>
    <b>Circumference of a circle is:</b>
    <%=getCircumference()%><br/>
    <hr/>
    <body></body>
</html>
```

JSP scriptlets

Summary

- JSP uses Java programming language and class libraries.
- JSP page uses HTML to display static text and Java code to generate dynamic content.
- Elements of JSP page are static content, JSP directives, JSP expressions, and JSP scriptlets.
- A JSP page can be created using standard development tools.
- JSP uses reusable and cross-platform components, such as JavaBeans.
- JSP allows the creation of user-defined tags and makes the JSP development process easy.
- Different phases in JSP life cycle are translation, compilation, and execution.